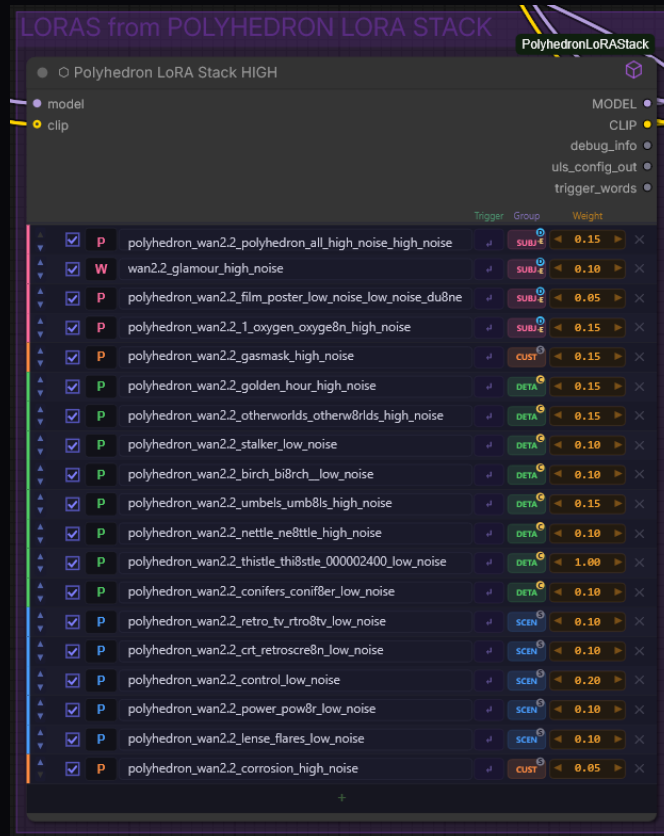


Polyhedron LoRA Stack

User Documentation · v124

by Polyhedron

A custom ComfyUI node for managing and applying large collections of LoRAs with group-based apply modes, cleanup switches, thumbnails, trigger words, an Accelerator engine and a Merge Analyzer.



Both Stack nodes side by side — HIGH and LOW noise paths in a WAN 2.2 dual-noise workflow.

© Polyhedron · All rights reserved

civitai.com/user/Polyhedron_AI · patreon.com/c/polyhedron_ai

1 · Overview

The Polyhedron LoRA Stack is a replacement for the standard Power LoRA Loader node. It is designed for workflows that use many LoRAs simultaneously — such as WAN 2.2 dual-noise image generation — where managing individual LoRA nodes becomes impractical.

The pack currently ships **thirteen nodes**. The three core nodes:

- **Polyhedron LoRA Stack** — the main node. Holds any number of LoRAs, organises them into groups, and applies them in a defined order.
- **Polyhedron LoRA Engine** — a companion node for acceleration LoRAs (LightX2V, FusionX, CausVid, ...). Flat list, single global apply mode, placed before the Stack in the pipeline.
- **Polyhedron LoRA Inspector** — a passive consistency-check node. Compares the active LoRAs from the Stack against the actual prompt text and reports which trigger words are present and which are missing.

New since this documentation was last updated:

- **Polyhedron Merge Analyzer** — a passive measurement node for the Combined / Smooth Mix merge path. Shows what the Stack is about to merge and, on request, measures how faithful the merge is (chapter 14).
- **Cleanup switches — Trim & Resolve** — two per-group refinement switches for stacks with many concurrent LoRAs (section 3.7).

The companion nodes — Token Counter, Select Model Switch, WAN Bridge, Frame Inflate / Pick Frame and the Sigma nodes — are covered in chapters 9–13.

Design principle

One node = one model. For dual-noise workflows, use two Stack nodes side by side (HIGH and LOW), each receiving its own model. The nodes are fully independent.

The backend is model-agnostic: FLUX, WAN 2.1, WAN 2.2, SDXL, SD 1.5 all work identically.

2 · Installation

Copy the folder into your ComfyUI custom nodes directory:

```
ComfyUI/custom_nodes/ComfyUI-PolyhedronLoRAStack/
```

Then install the dependencies. For standard Python environments:

```
pip install -r requirements.txt
```

For ComfyUI Portable (Windows):

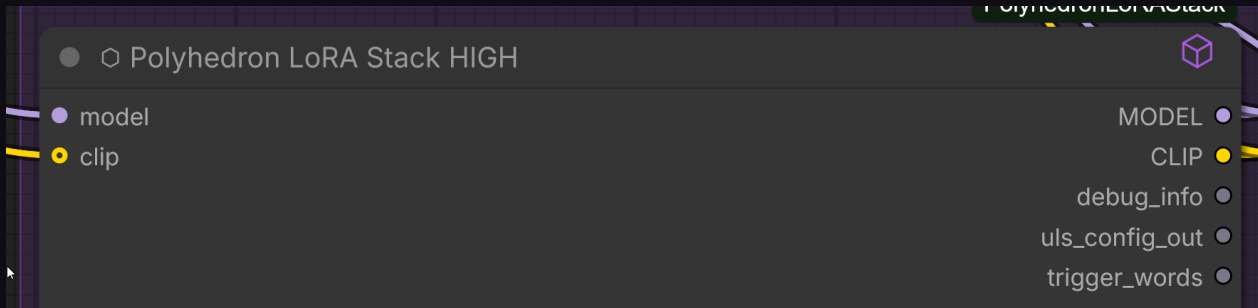
```
D:\ComfyUI\python_embeded\python.exe -m pip install -r requirements.txt
```

Dependencies: Pillow >= 9.0.0, requests >= 2.28.0

3 · Polyhedron LoRA Stack

The Stack node accepts a MODEL and optionally a CLIP input, applies all active LoRAs in group order, and outputs the patched model along with debug information and collected trigger words.

3.1 · Node header

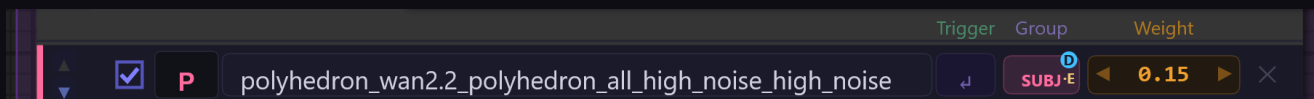


Stack node header — inputs on the left, outputs on the right.

The header contains:

- **model** (input) — the diffusion model to patch.
- **clip** (input, optional) — the text encoder. Not required for WAN/video models.
- **MODEL** (output) — the patched model, ready for the sampler.
- **CLIP** (output) — the patched text encoder.
- **debug_info** (output) — lists every applied LoRA, weight, group and mode.
- **uls_config_out** (output) — internal configuration passthrough for chaining and analysis.
- **trigger_words** (output) — all trigger words from active LoRAs, comma-separated.

3.2 · Row anatomy

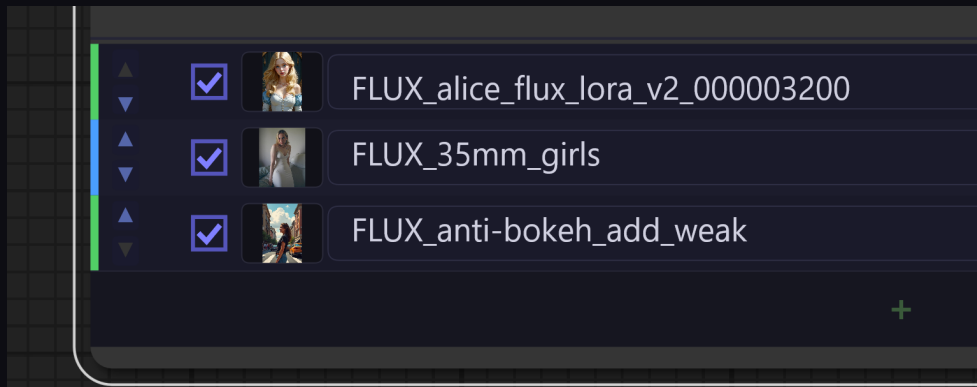


A single LoRA row — all interactive elements visible.

Each row represents one LoRA. From left to right:

- ▲ ▼ — move the row up or down in the list.
- **Checkbox** — enable or disable this LoRA without removing it.
- **Thumbnail** — preview image if available. Click to open the detail overlay.
- **Name button** — displays the filename. Click to open the LoRA selector with search.
- ↵ **Insert** — inserts trigger word(s) at the cursor in the active text node.
- **Group pill** — shows the group (SUBJ, DETA, SCEN ...). Left-click cycles groups. Right-click opens the apply mode panel.
- ◀ **Weight** ▶ — LoRA strength. Click arrows to step by 0.05, or click the number to type directly.
- ✕ — remove this row.

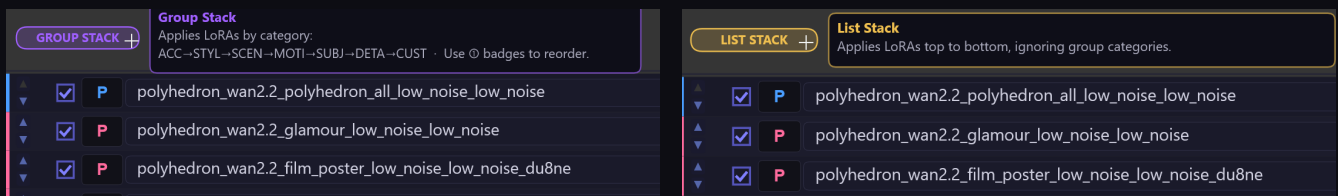
3.3 · Adding and reordering rows



▲▼ reorder buttons and the + button to add new rows.

Click + at the bottom of the list to add an empty row. Use ▲ / ▼ to reorder, or drag from the far-left handle.

3.4 · Stack mode and group order



GROUP STACK — LoRAs applied by category in a fixed order. · LIST STACK — LoRAs applied top to bottom, ignoring groups.

The **GROUP STACK** / **LIST STACK** pill in the header controls how LoRAs are ordered when applied. Click it to toggle between the two modes.

- **GROUP STACK** (purple) — LoRAs are applied in group order: ACC → STYL → SCEN → MOTI → SUBJ → DETA → CUST. This is the default and recommended mode.
- **LIST STACK** (gold) — LoRAs are applied exactly in the order they appear in the list, top to bottom. Groups are ignored. Useful for quick testing or when list order matters more than categories.

Custom group order

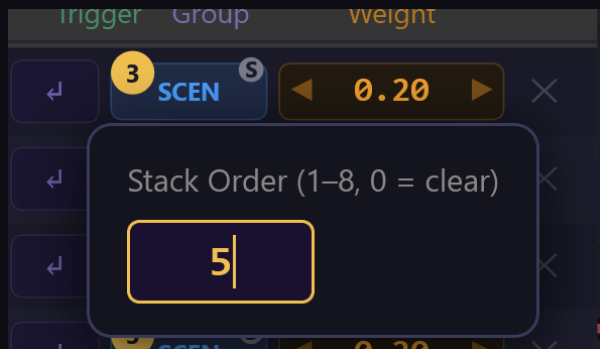


Gold badge = custom order assigned. · Dashed circle = no order set (runs after numbered groups).

In **GROUP STACK** mode, each group can be assigned a custom stack position using the **gold order badge** in the top-left corner of the Group pill.

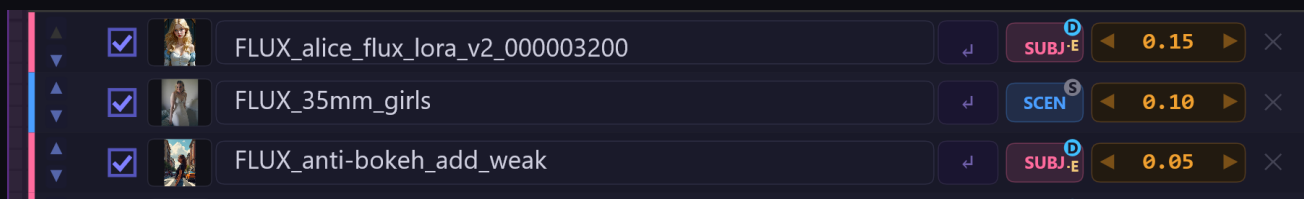
- Gold filled badge with a number — this group runs at that position.

- Dashed circle — no position set, runs after all numbered groups.
- **Click the badge** to open the Stack Order input. Enter 1–8 to assign a position, 0 to clear.
- Each number can only be used once — a duplicate entry is rejected and the conflicting badge flashes red briefly.



Stack Order input — integer 1–8, enter 0 to clear.

3.5 · Groups



Group pills: SUBJ·E = Element Drop, SCEN·S = Sequential, DETA·C = Channel Drop.

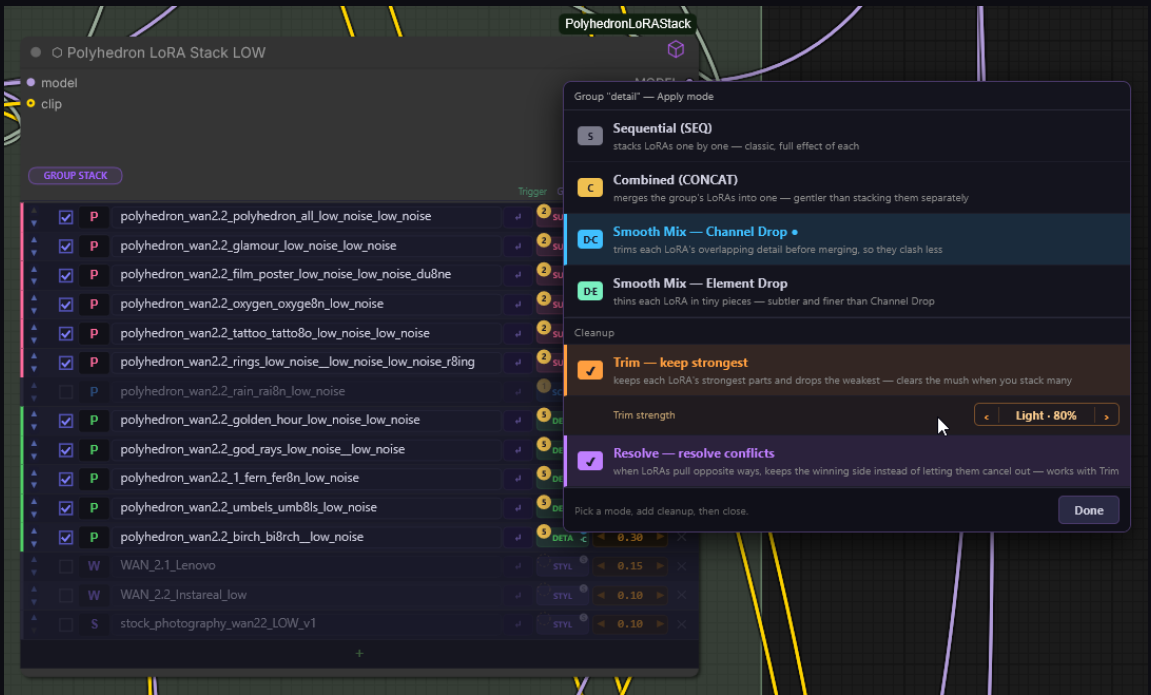
Every LoRA belongs to a group. Groups control the *application order* and the *apply mode* for that batch. Default group order:

ACC → STYL → SCEN → MOTI → SUBJ → DETA → CUST → (ungrouped)

Group	Badge	Intended content	Default mode
—	—	Ungrouped / unassigned LoRAs	SEQ
acc	ACC	Acceleration / distillation LoRAs (LightX2V, FusionX, CausVid ...)	SEQ
style	STYL	Film look, colour grading, artist styles	SEQ
scene	SCEN	Setting, atmosphere, vegetation, objects	SEQ
motion	MOTI	WAN motion LoRAs	SEQ
subject	SUBJ	Main subject, clothing, accessories	SEQ
detail	DETA	Skin, eyes, hands, anatomy corrections	SEQ
custom	CUST	Anything else	SEQ

Note: Group assignments persist in `uls_groups.json` next to your LoRAs folder — they survive workflow reloads and ComfyUI restarts.

3.6 · Apply modes



Right-click on any Group pill to open the apply-mode panel — base modes on top, Cleanup switches below.

Each group has its own apply mode. **Right-click the Group pill** to open the apply-mode panel. The mode applies to all rows in that group.

The panel **stays open** while you click, so you can pick a base mode and layer the Cleanup switches (section 3.7) in one go. Close it with **Done**, a click outside, or **Escape**.

Mode	Badge	What it does	Best for
Sequential	S (grey)	Applies each LoRA one by one — classic, full effect of each.	Default. Always safe.
Combined	C (gold)	Merges the group's LoRAs into one patch — gentler than stacking them separately.	Experimental comparison with SEQ.
Smooth Mix — Channel Drop	D·C (cyan/green)	Like Combined, but trims each LoRA's overlapping detail before merging — drops whole rank-channels, LoRA-aware.	Groups with many LoRAs targeting the same area.
Smooth Mix — Element Drop	D·E (cyan/gold)	Like Combined, but thins each LoRA in tiny pieces — drops individual tensor elements (classic DARE paper).	Alternative to Channel Drop — finer-grained.

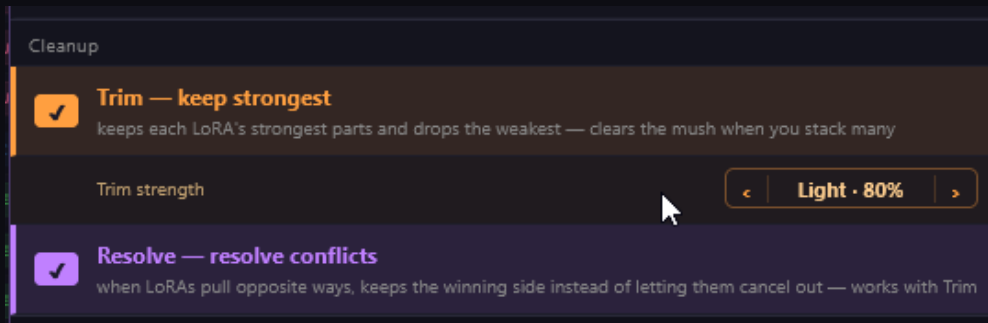
DARE density — automatic scaling

$$\text{density} = \max(0.5, 1 - 0.05 \times (n-1)) \quad | \quad 2 \text{ LoRAs: } 0.95 \quad | \quad 5 \text{ LoRAs: } 0.80 \quad | \quad 10 \text{ LoRAs: } 0.55$$

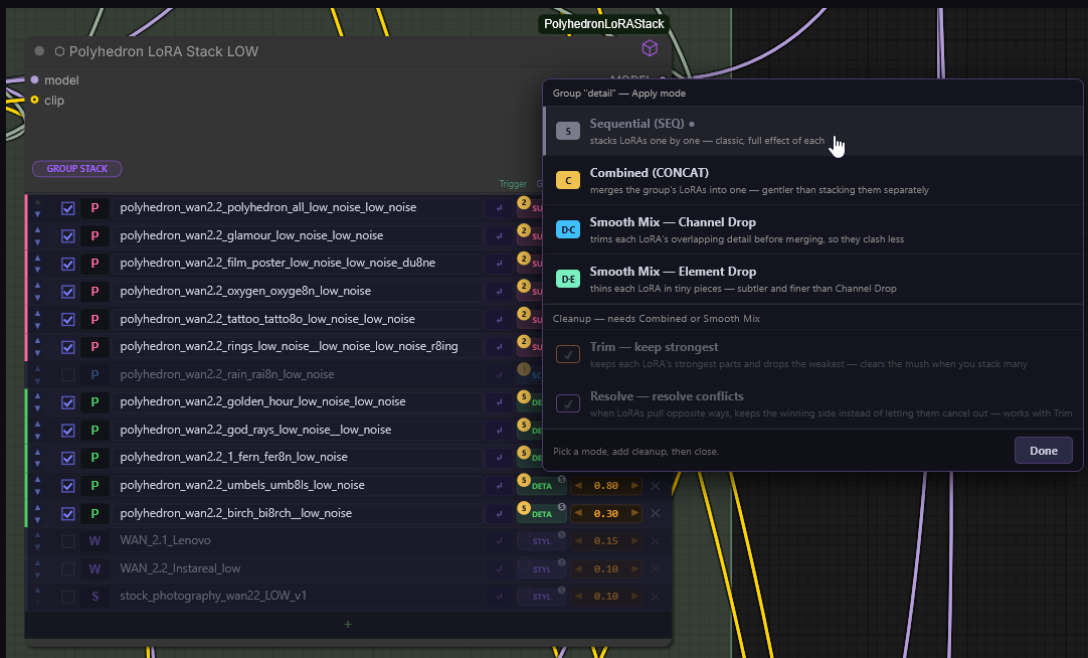
The DARE seed is deterministic (SHA-1 of names + weights) — same config = same result every session.

3.7 · Cleanup switches — Trim & Resolve

Below the four base modes, the panel has a **Cleanup** section with two optional switches. They refine the **Combined / Smooth Mix** merge for groups with many concurrent LoRAs. In **Sequential** mode the section is greyed out ("Cleanup — needs Combined or Smooth Mix") and the backend ignores both switches.



The Cleanup section — Trim with its strength stepper, and Resolve.



On a Sequential group the Cleanup section is greyed out — switch states are kept, but ignored.

Trim — keep strongest

Keeps each LoRA's strongest parts and drops the weakest — clears the "mush" that builds up when many LoRAs are stacked at once. The trim is **deterministic** (no randomness): for each LoRA, the weakest rank-channels are removed by magnitude *before* the group is merged.

Trim strength — a stepper under the Trim switch (visible while Trim is on) sets how much of each LoRA is kept:

Step	Kept share	Character
Auto	group-scaled	recommended starting point
Gentle	90%	barely noticeable
Light	80%	safe everyday value
Medium	70%	noticeable cleanup
Strong	60%	aggressive
Max	50%	extreme stacks only — see warning

◀ steps toward Auto (gentler), ▶ toward Max (stronger).

Auto scaling

The more LoRAs are active in the group, the more of the weak tail is pruned: 2 LoRAs keep ~95%, 5 keep ~88%, 10 keep ~73%, 15+ hit the floor of 60%.

⚠ Too much Trim

Max · 50% can cut away the very channels that make a concept distinctive: the subject collapses to a silhouette and the base model fills the gap with unprompted content. A high LoRA weight does **not** protect against this — the trim ranking runs inside each LoRA. Use Max only for extreme stacks (15+ LoRAs) and always check results visually. If renders look washed out with Trim on, lower the Trim strength first. The recommended working range is **70-80%** (Medium/Light); the Merge Analyzer (chapter 14) warns about this in its report.

Resolve — resolve conflicts

When LoRAs pull in opposite directions, Resolve keeps the winning side instead of letting them cancel each other out — and it works together with Trim. Under the hood it runs a sign election across the group's competing LoRAs and then re-packs the result so it stays as light as a normal LoRA patch.

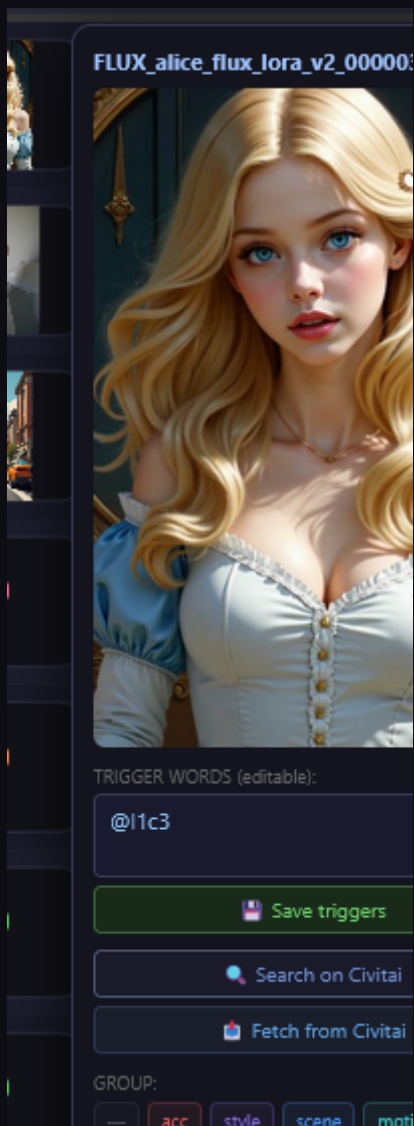
Good to know

- **Persistent** — the switch states *and* the Trim strength are saved into the workflow and restored on load. They survive ComfyUI restarts and never reset to defaults on their own. (Group *assignments* persist separately in `uls_groups.json`, see section 3.5.)
- **Visible in debug_info** — each group is tagged with its full mode, e.g. `DARE [CHAN] +TRIM +RESOLVE`.
- **Console feedback** — long Combined / Smooth Mix merges print a live progress log to the console (`[PLS]` lines, plus a per-phase merge-timing summary at the end), so you can see what a long merge is doing.
- **Cancel works** — ComfyUI's red × aborts a running merge promptly instead of letting it finish in the background.


```
[PLS] RESOLVE device: cuda [fp16 matmuls, fp32 select+SVD]
[PLS] TRIM: keep fraction=0.850 n=6 (dropping weakest rank-channels, deterministic)
[PLS] RESOLVE 1/400 cuda/fp16 layer=0.15s cum=0.1s free=13.6G
[PLS] RESOLVE 25/400 cuda/fp16 layer=0.17s cum=2.6s free=11.9G
[PLS] RESOLVE 50/400 cuda/fp16 layer=0.09s cum=5.1s free=11.9G
[PLS] RESOLVE 75/400 cuda/fp16 layer=0.18s cum=7.6s free=11.9G
[PLS] RESOLVE 100/400 cuda/fp16 layer=0.08s cum=10.1s free=11.9G
[PLS] RESOLVE 125/400 cuda/fp16 layer=0.17s cum=12.6s free=11.9G
[PLS] RESOLVE 150/400 cuda/fp16 layer=0.08s cum=15.1s free=11.9G
[PLS] RESOLVE 175/400 cuda/fp16 layer=0.17s cum=17.6s free=11.9G
[PLS] RESOLVE 200/400 cuda/fp16 layer=0.08s cum=20.1s free=11.9G
[PLS] RESOLVE 225/400 cuda/fp16 layer=0.17s cum=22.5s free=11.9G
[PLS] RESOLVE 250/400 cuda/fp16 layer=0.08s cum=25.1s free=11.9G
[PLS] RESOLVE 275/400 cuda/fp16 layer=0.18s cum=27.7s free=11.9G
[PLS] RESOLVE 300/400 cuda/fp16 layer=0.08s cum=30.3s free=11.9G
[PLS] RESOLVE 325/400 cuda/fp16 layer=0.20s cum=33.0s free=11.9G
[PLS] RESOLVE 350/400 cuda/fp16 layer=0.10s cum=35.7s free=11.9G
[PLS] RESOLVE 375/400 cuda/fp16 layer=0.19s cum=38.5s free=11.9G
[PLS] RESOLVE 400/400 cuda/fp16 layer=0.09s cum=41.2s free=11.9G
[PLS] CONCAT: 2400 layer(s) had no alpha + assumed scale=1.0
[PLS] TRIM: kept 750/89000 rank-channels (dropped 14000 weakest across all sources)
[PLS] RESOLVE: sign-election + disjoint merge over 6 LoRAs (after trim): resolved delta re-packed low-rank per layer
[PLS] ✓ CONCAT +TRIM +RESOLVE merged 6 LoRAs [polyhedron_wan2.2_, wan2.2_glamour_hig, polyhedron_wan2.2_, polyhedron_wan2.2_, polyhedron_wan2.2_] layers=400 patches=400
[PLS] ⚡ merge timing CONCAT +TRIM +RESOLVE n=6, 400 layers - CPU, once (before the sampler)
[PLS] load 0.17s 0 % safetensors read (cached on re-run)
[PLS] trim 1.96s 4 % magnitude top-k
[PLS] resolve 41.22s 88 % TIES + SVD x400 layers [cuda/fp16]
[PLS] other 3.68s 8 % concat + DARE mask + hand-off
[PLS] total 47.02s
```

Toggle the Smooth Mix variant by clicking the ·C/E badge, or via the right-click panel. The pill shows the **base mode** only — the Cleanup switches are visible in the right-click panel and in `debug_info` (`+TRIM` / `+RESOLVE`).

3.9 · Thumbnail overlay



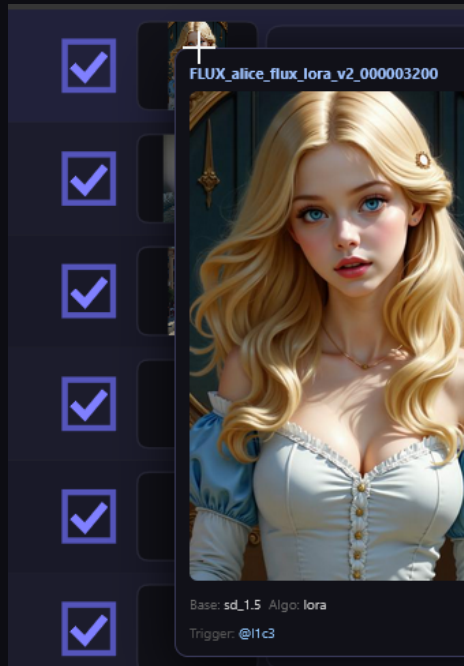
Thumbnail overlay — preview, trigger words, Civitai, group selector, Smooth Mix variant.

Click a thumbnail to open the detail overlay:

- **Preview image** — LoRA preview if available.
- **TRIGGER WORDS** — editable. Click Save triggers to store.
- **Search on Civitai / Fetch from Civitai** — Civitai integration.
- **GROUP** — reassign the LoRA to a different group.
- **DARE VARIANT** — only visible when the group uses Smooth Mix. Select CHAN or ELEM.

Note: Trigger .txt files are read-only. Edits are stored in .uls-meta.json.

3.10 · Hover preview



Hovering over a thumbnail shows a quick preview popup.

Hovering over a thumbnail shows a popup with preview image, base model type, and trigger words — no click needed.

3.11 · Trigger word insertion

The ↵ button inserts trigger word(s) at the cursor in the active text node:

- **Single trigger** — inserted immediately with a toast confirmation.
- **Multiple triggers** — a selection popup lists all options. Click one to insert.



Trigger selection popup. · Confirmation toast + trigger in the prompt node.

3.12 · Setting weights

Three ways to change a LoRA's weight:

- ◀/▶ — step by 0.05.
- **Click the number** — type any value, press Enter. Range: -10.0 to 10.0.

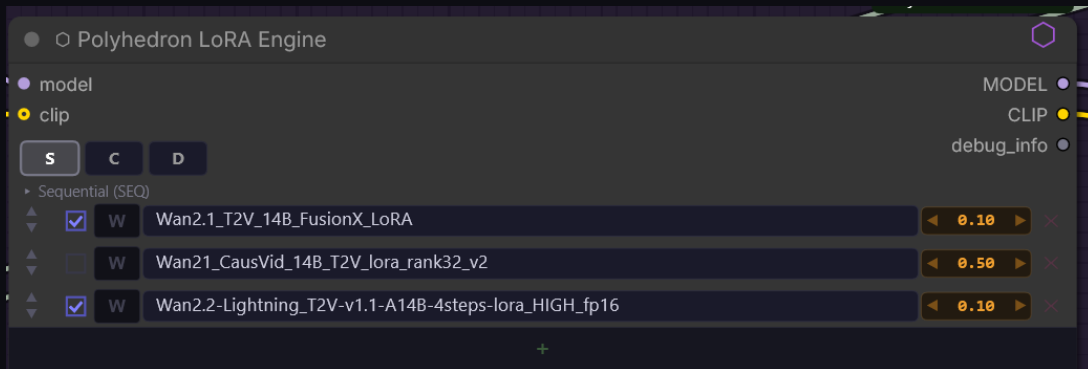


Direct weight input field.

4 · Polyhedron LoRA Engine

The Engine node is designed for **acceleration LoRAs** — LoRAs that modify *how* the model generates (inference trajectory, step reduction) rather than *what* it depicts. Examples: LightX2V, FusionX, CausVid, WAN 2.1 Lightning.

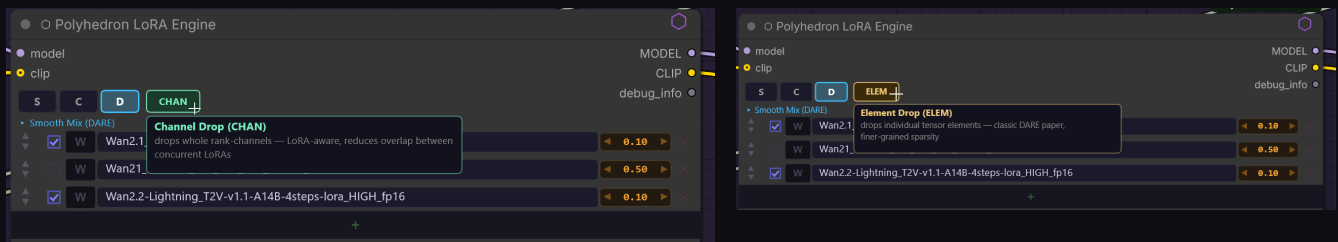
It uses a flat list with a single global apply mode. Place it *before* the Stack node — connect MODEL and CLIP outputs of the Engine into the Stack inputs.



Engine node — S mode active, three acceleration LoRAs loaded.

4.1 · Mode buttons and DARE variant

Buttons **S**, **C**, **D** select the apply mode for all Engine LoRAs. When **D** is active, a **CHAN** or **ELEM** pill appears to its right. Click to toggle. Hover for a tooltip.



CHAN — Channel Drop tooltip. · ELEM — Element Drop tooltip.

The Cleanup switches (section 3.7) are currently **Stack-only**; the Engine offers the plain S / C / D modes.

Engine vs Stack — when to use Smooth Mix

For acceleration LoRAs (typically 2-3), SEQ is the recommended default.

Smooth Mix is most useful in the Stack when a group has 5+ LoRAs targeting the same model areas.

5 · Polyhedron LoRA Inspector

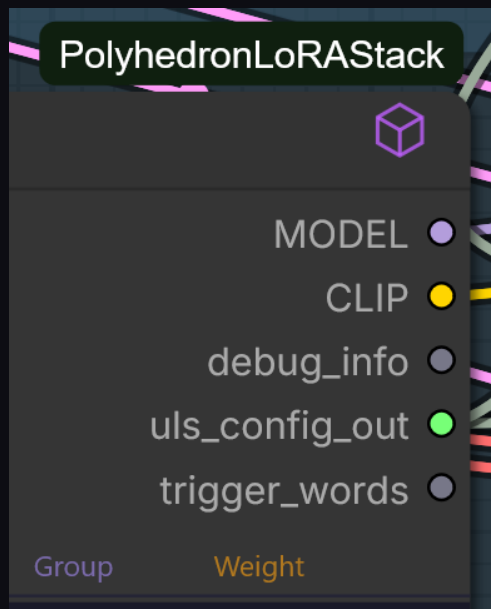
The Inspector node is a **passive consistency-check tool**. It compares the active LoRAs from the Stack node against the actual text in your prompt and reports which trigger words are present and which are missing — without modifying anything.

This is useful when working with many LoRAs at small weights: trigger words are easy to forget, and a missing trigger silently weakens the LoRA's effect. The Inspector makes this visible at a glance.

5.1 · Inputs and output

The Inspector has two inputs and one output:

- **uls_config_out** (input, green link) — connect from the Stack's *uls_config_out* output. Carries the list of active LoRAs with their weights, groups, and trigger words.
- **prompt** (input, STRING) — connect the same string that goes into your CLIPTextEncode / prompt encoder.
- **inspector_report** (output, STRING) — formatted report. Connect to a Show Text node to display it.



*Stack node — green *uls_config_out* output pin connects to the Inspector.*

5.2 · How matching works

The Inspector checks each active LoRA's trigger words against the prompt:

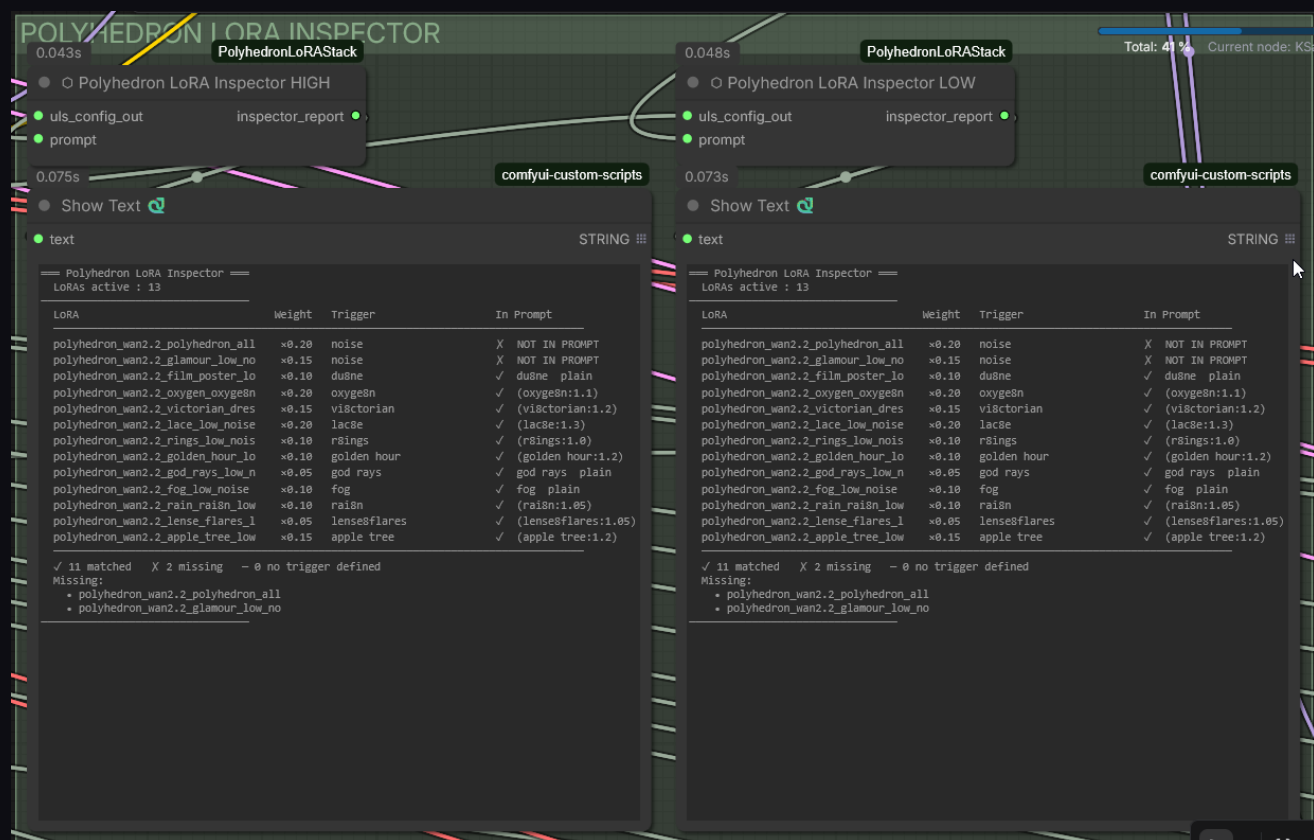
- **Weighted syntax** — (trigger:1.2), (trigger:0.8) etc. is recognised and shown verbatim in the report.
- **Plain keywords** — bare trigger words are matched with word boundaries, so "ring" inside "stinging" is not a false positive.
- **No trigger defined** — LoRAs without trigger words are listed separately and don't count as missing.

Trigger word resolution priority

1. .uls-meta.json (user-edited triggers from the thumbnail overlay)
2. .txt file next to the .safetensors (read-only)
3. Embedded safetensors header metadata
4. Filename fallback

The Inspector uses the same priority as the Stack — what you see in the overlay is what the Inspector matches against.

5.3 • Report format



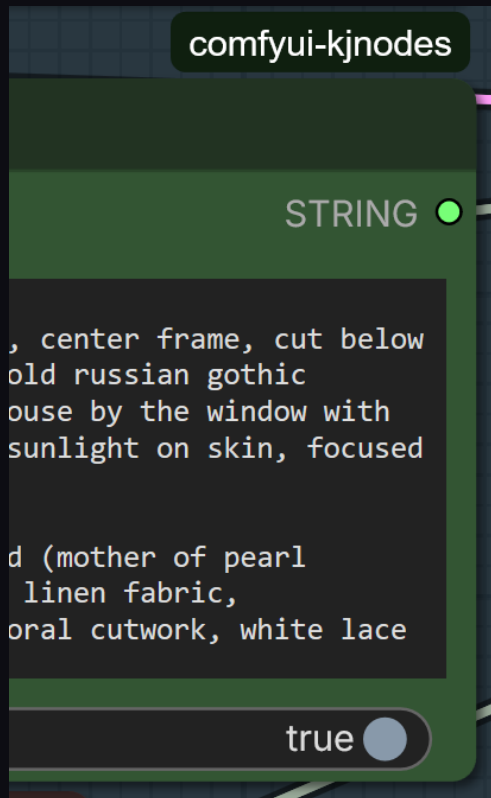
Two Inspector nodes for HIGH and LOW Stacks — same prompt input, different uls_config_out links.

The report is a fixed-width text table:

- **LoRA** — short filename
- **Weight** — current weight in the Stack
- **Trigger** — the trigger word(s) defined for this LoRA
- **In Prompt** — ✓ if found (with the actual matched syntax shown), X if missing, or 'plain' if found as a plain keyword without weight syntax

A summary line at the bottom shows totals: matched, missing, no trigger defined. Missing LoRAs are listed by name so they're easy to fix.

5.4 · Dual-noise pattern



The prompt String node — feeds both Inspector nodes and the CLIPTextEncode.

For dual-noise workflows (WAN 2.2): use **two Inspector nodes**, one connected to the HIGH Stack's *uls_config_out* and one to the LOW Stack. Both receive the same prompt. This way you can verify that all triggers for both noise paths are present in the prompt.

Inspector is passive

The Inspector never modifies your prompt or your LoRA configuration. It only reads and reports. Disconnect it any time without consequences.

The Show Text node connected to *inspector_report* only runs when needed — the Inspector itself doesn't trigger workflow execution.

6 · Connecting the outputs

Stack node outputs and how to connect them:

- **MODEL** → sampler (KSampler, KSamplerAdvanced, etc.)
- **CLIP** → CLIPTextEncode nodes
- **debug_info** → any text display node (optional but useful during setup)
- **uls_config_out** → Polyhedron LoRA Inspector (chapter 5) and/or Merge Analyzer (chapter 14); also used when chaining Stack nodes. Otherwise leave unconnected.
- **trigger_words** → JoinStrings or directly into your positive prompt node

Trigger words output

The trigger_words output collects trigger words from all active LoRAs automatically. Connect it to JoinStrings together with your manual prompt. No manual trigger word management needed as LoRAs are added or removed.

7 · Tips and recommended settings

7.1 · Weight strategy

The Polyhedron workflow uses many LoRAs at small weights (0.05–0.20) rather than few LoRAs at full strength:

- Averages across multiple training domains — reduces LoRA burn
- Allows fine-grained control over each element's contribution
- Makes Smooth Mix subtle at low weights — acts as insurance rather than a strong effect

7.2 · Dual-noise workflow (WAN 2.2)

Use two Stack nodes side by side — one for HIGH noise, one for LOW noise. Each receives its own separate WAN 2.2 model. Configure both identically.

7.3 · Group mode recommendations

Group	LoRAs	Recommended mode	Reason
subject (SUBJ)	3–5	SEQ	Full individual effect needed
detail (DETA)	5–8	Smooth Mix · CHAN + Trim + Resolve	Many overlapping LoRAs — classic conflict case
scene (SCEN)	3–5	SEQ	Small enough that conflicts are minimal
acc (ACC)	2–3	SEQ	Acceleration LoRAs rarely conflict

For Trim, start with **Auto** and stay in the **70–80%** corridor when overriding — see the warning in section 3.7 before going lower.

8 · Preview images and trigger words

Place files alongside your .safetensors files:

- **Preview image:** same base name, .jpg extension
- **Trigger words:** same base name, .txt extension. Read-only — never modified.
- Edits made in the overlay are stored in .uls-meta.json (auto-created). These take priority over the .txt file if both exist.
- Trigger words also read from the safetensors header as fallback.

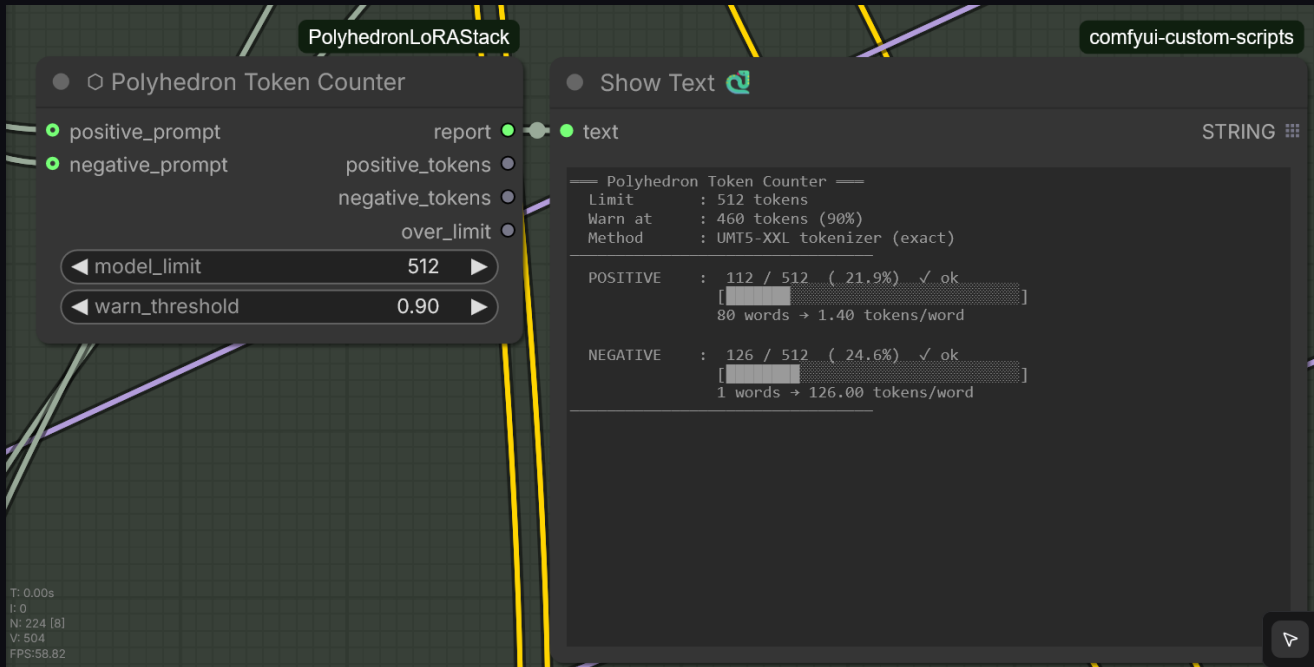
Resolution priority (highest first): `.uls-meta.json` → `.txt` → `safetensors header` → `filename`

Note: Civitai preview images and trigger words can be fetched automatically via "Fetch from Civitai" in the thumbnail overlay.

Note: Civitai preview selection is SFW-strict — only gallery images rated None/Soft are used as previews. If a LoRA's Civitai gallery contains exclusively Mature/X-rated images, no preview is stored.

9 · Polyhedron Token Counter

The Token Counter is a companion node for the Stack. It reads both the positive and negative prompt strings and counts how many tokens each consumes — measured against the limit of the active text encoder. When working with large LoRA collections, trigger words from 15–20 active LoRAs can silently push the prompt toward the encoder limit, degrading output quality without any warning in ComfyUI itself.



Token Counter node connected to a Show Text node. The report shows POSITIVE and NEGATIVE prompts separately with usage bars, percentages, and tokens-per-word ratios.

9.1 · Inputs, outputs and widgets

Name	Type	Description
positive_prompt	STRING (input)	Connect from your positive prompt node or from trigger_words.
negative_prompt	STRING (input)	Connect from your negative prompt node.
report	STRING (output)	Full formatted report — connect to a Show Text node.
positive_tokens	INT (output)	Raw token count for the positive prompt.
negative_tokens	INT (output)	Raw token count for the negative prompt.
over_limit	BOOLEAN (output)	True when either prompt exceeds warn_threshold × model_limit.
model_limit	INT (widget)	Token limit of the active encoder. Default 512 (UMT5 / WAN).
warn_threshold	FLOAT (widget)	Fraction of model_limit at which to warn. Default 0.90 = warn at 90%.

9.2 · Report output

The report shows POSITIVE and NEGATIVE separately, each with a usage bar, percentage, word count, and a tokens-per-word ratio. A checkmark (ok) appears when under the warn threshold; an exclamation mark appears when the threshold is exceeded.

Typical placement

Connect positive_prompt from the same string node that feeds your CLIPTextEncode. Connect negative_prompt from the negative string. The node runs passively — it never modifies data, only counts.

9.3 · Token limits by model

Different text encoders have different token limits. The table below covers all models commonly used in ComfyUI workflows. The WAN row (highlighted) is the primary target of this plugin.

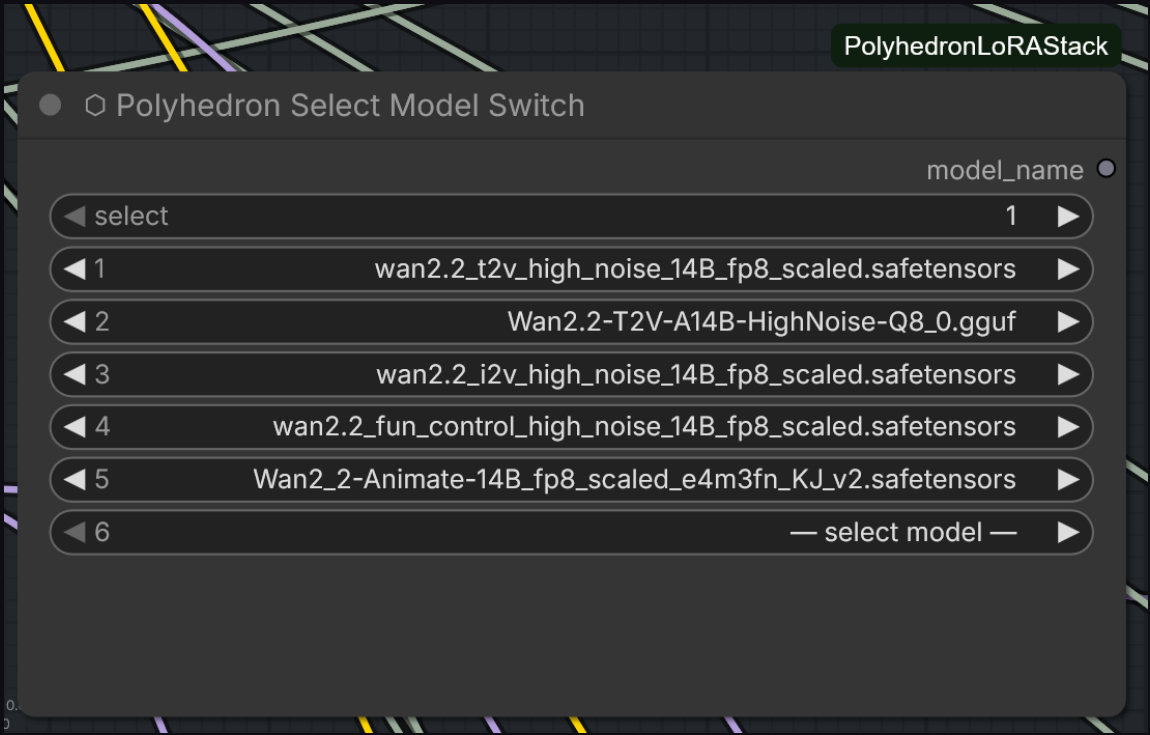
Model	Text Encoder(s)	Hard Limit	Practical	Notes
SD 1.5	CLIP-L	77	75	One chunk = 75 usable tokens. ComfyUI can chain chunks.
SDXL	CLIP-L + CLIP-G	77 each	75 each	Two encoders in parallel. Long-prompt extensions available.
SD 3 / 3.5	CLIP-L + CLIP-G + T5-XXL	77/77/512	75/75/512	Triple encoder. T5 carries most semantic load.
FLUX.1 dev / schnell	CLIP-L + T5-XXL	77 / 512	75 / 512	CLIP-L: style anchor. T5-XXL: full prompt detail.
FLUX.1 Kontext	CLIP-L + T5-XXL	77 / 512	75 / 512	Same encoders as FLUX.1. Image tokens add to budget.
FLUX.2 / Lumina	CLIP-L + Mistral-3 24B	77 / ~4096	75 / ~2000	Mistral-3 replaces T5. Quality gain plateaus early.
Qwen-Image	Qwen2.5-VL 7B	No hard limit	~500	LLM-based encoder. Practically limited by VRAM and quality.
WAN 2.1 / 2.2	UMT5-XXL	512	~256-300	Hard limit 512. Quality degrades above ~320-350: slower motion, grid artefacts.
HunyuanVideo / 1.5	LLaMA-3 + CLIP-L	1000 / 77	~400 / 75	LLaMA-3 is primary (1000 limit, ~400 recommended). CLIP-L for visual anchor.
CogVideoX	T5-XXL	226	200	T5 capped at 226 in official implementation.
Mochi / LTX-Video	T5-XXL	256	200	T5-XXL at 256 tokens.

WAN / UMT5 practical limit

Although UMT5 supports up to 512 tokens, quality degrades noticeably above approximately 320–350 tokens: motion slows down and a grid pattern artefact can appear in the output video. The default warn_threshold of 0.90 (= 460 tokens) is conservative — consider lowering it to 0.60–0.65 (~300–330 tokens) for WAN workflows.

10 · Polyhedron Select Model Switch

The Select Model Switch node provides six model slots and outputs the filename of the active model as a string. It is designed to sit next to your model loader nodes, allowing fast switching between model variants without rewiring the graph.



The Polyhedron Select Model Switch node with all six slots populated. Slots 1 and 2 are wire inputs (green dots on the left); slots 3-6 are dropdown widgets built directly into the node.

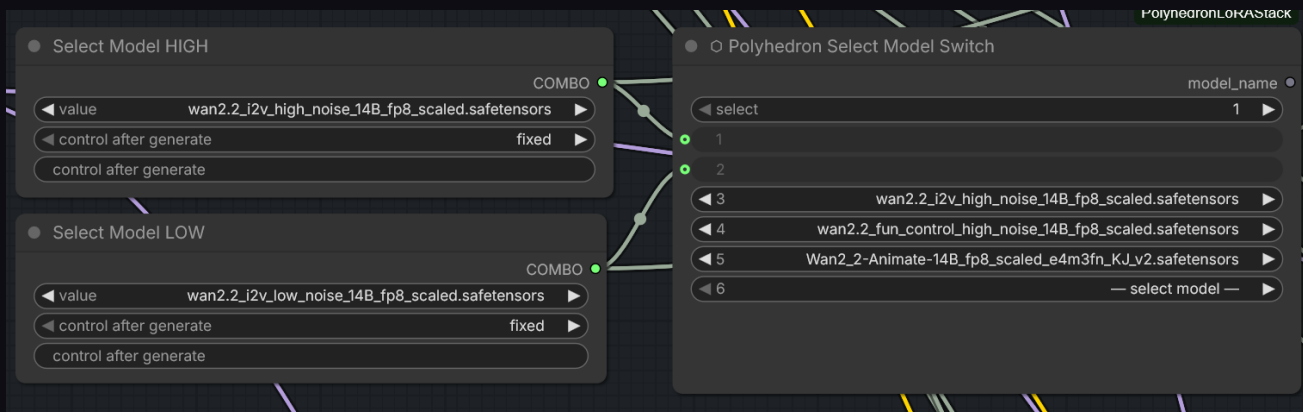
Slots 1 and 2 are **wire inputs** (green dots) — connect a MODEL output from any loader node. Slots 3-6 are **dropdown widgets** built directly into the node — click the arrow to open a searchable list of all models in your `diffusion_models` folder.

10.1 · Inputs, outputs and widgets

Name	Type	Description
1, 2	MODEL (input)	Wire inputs. Connect any loader node output directly.
3, 4, 5, 6	COMBO (widget)	Dropdown selectors. Choose a model filename from the list.
select	INT (widget)	Active slot index (1-6). Only this slot's model is used.
model_name	STRING (output)	Filename of the active model. Connect to a model loader's path input.

10.2 · Wire-input usage

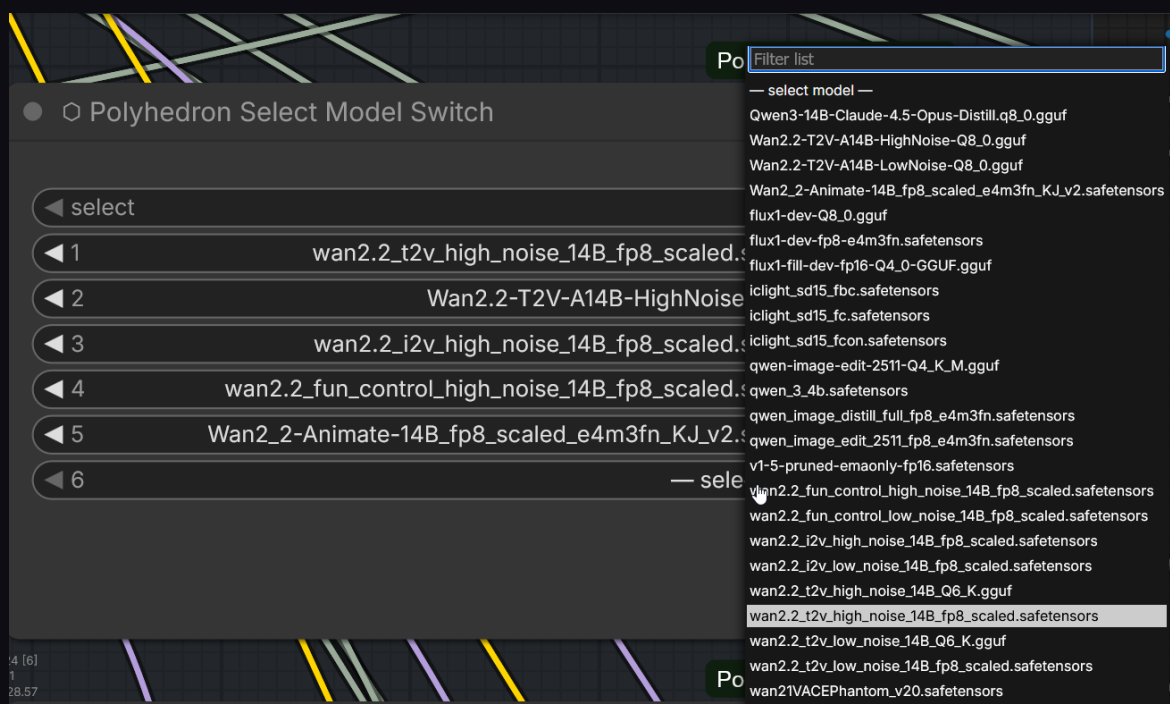
Slots 1 and 2 accept any MODEL connection. A typical use is to wire two model loader nodes — one HIGH and one LOW — directly into the Switch node, with the slot dropdowns 3-6 holding additional variants for quick comparison without rewiring.



Two Select Model nodes (left) wired into a Polyhedron Select Model Switch (right). The Switch holds additional variants in slots 3-5; slot 6 is left as the placeholder fallback.

10.3 · Searchable dropdown

Each dropdown slot (3-6) opens a filterable list of all models in the `diffusion_models` folder. Start typing in the filter field at the top to narrow the list — particularly useful when you have dozens of model variants installed.



The dropdown selector with a filter field at the top. All models in the `diffusion_models` folder are listed and can be searched.

10.4 · Slot 6 placeholder

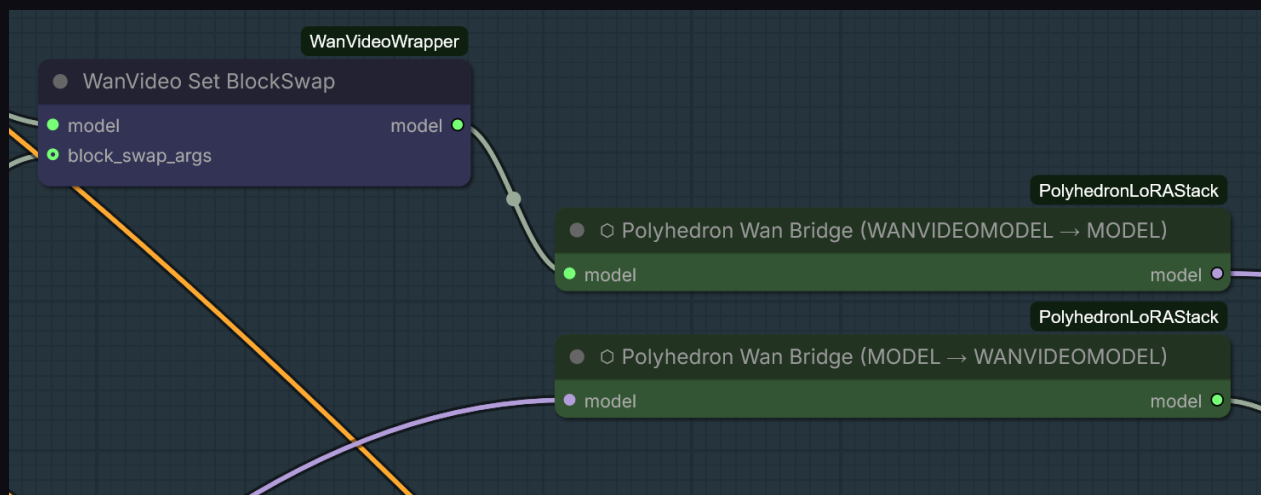
Slot 6 defaults to **— select model —**. Leave it as-is to use it as an intentionally empty fallback, or assign a model to use all six slots.

10.5 · Recommended use in dual-noise workflows

In a WAN 2.2 dual-noise setup, place one Switch node on the HIGH path and one on the LOW path. Wire or select all your HIGH model variants in the first Switch and all LOW variants in the second. Change the select index on both to swap the full model pair in a single edit.

11 · Polyhedron WAN Bridge

ComfyUI uses two distinct model types for WAN workflows: **WANVIDEOMODEL** (the native type used by WanVideoWrapper nodes) and **MODEL** (the standard ComfyUI type used by KSampler, LoRA loaders, and most other nodes). The two Bridge nodes convert between these types so that both ecosystems can be used in the same pipeline.



Both Bridge variants side by side. Top: WANVIDEOMODEL → MODEL (green input dot, lilac output dot). Bottom: MODEL → WANVIDEOMODEL (lilac input dot, green output dot). The colour of the dots indicates the type at that connection point.

Polyhedron Wan Bridge (WANVIDEOMODEL → MODEL)

Takes a WANVIDEOMODEL input and outputs a standard MODEL. Use this when you want to pass a WanVideoWrapper-loaded model into nodes that expect the standard MODEL type — for example the Polyhedron LoRA Stack or a KSampler.

Polyhedron Wan Bridge (MODEL → WANVIDEOMODEL)

Takes a standard MODEL input and outputs a WANVIDEOMODEL. Use this to pass a model that has been processed by standard ComfyUI nodes (e.g. after LoRA patching) back into WanVideoWrapper nodes.

11.1 · Inputs and outputs (both nodes)

Name	Type	Description
model	WANVIDEOMODEL or MODEL (input)	The model to convert. Type depends on which Bridge variant is used.
model	MODEL or WANVIDEOMODEL (output)	The converted model. Passed downstream to the next node.

11.2 · Typical pipeline position

In the standard dual-noise workflow:

- WanVideo model loader → **Bridge (WANVIDEOMODEL → MODEL)** → LoRA Stack → WanMoeKSampler
- LoRA Stack (MODEL output) → **Bridge (MODEL → WANVIDEOMODEL)** → WanVideoWrapper sampler

Stability note

Both Bridge nodes are pure type-cast operations — they do not modify model weights or patches. They have been stable since v232 and are not modified during active development.

12 · Polyhedron Wan Frame Inflate (T2I LoRA fix)

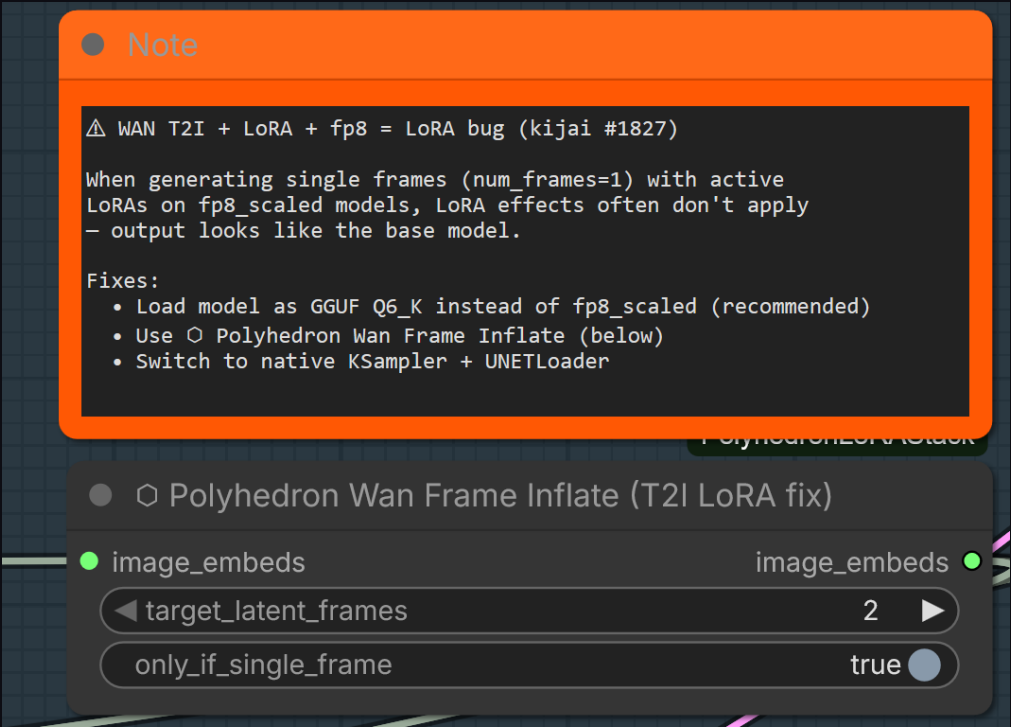
WAN T2I + LoRA + fp8 = LoRA bug (kijai #1827)

When generating single frames (num_frames=1) with active LoRAs on fp8_scaled models, LoRA effects often do not apply — the output looks like the base model without any LoRA influence.

Available fixes:

- Load model as GGUF Q6_K instead of fp8_scaled (**recommended**)
- Use the Polyhedron Wan Frame Inflate node (below)
- Switch to native KSampler + UNETLoader

The Frame Inflate node works by temporarily inflating the **image_embeds** tensor to a multi-frame sequence before sampling, which forces the fp8 model to activate its LoRA-aware code path. After sampling, the extra frames are discarded.



The Frame Inflate node together with the in-workflow note that documents the underlying bug and the three available workarounds.

12.1 · Inputs, outputs and widgets

Name	Type	Description
image_embeds	IMAGE_EMBEDS (input)	Connect from the image embedding node upstream of the sampler.
target_latent_frames	INT (widget)	Number of frames to inflate to. Default: 2.
only_if_single_frame	BOOLEAN (widget)	When true, only inflates if the input is a single frame. Default: true.
image_embeds	IMAGE_EMBEDS (output)	Inflated tensor, ready for the sampler.

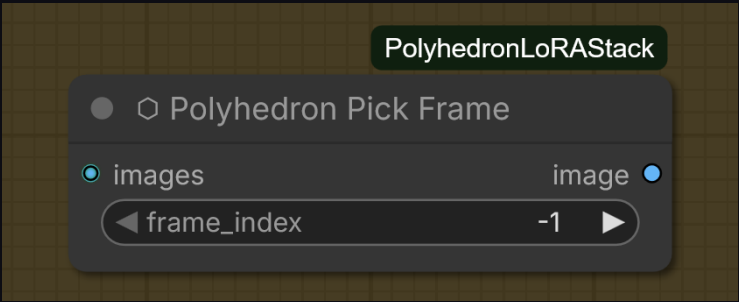
12.2 · When to use

- You are generating single still frames (T2I mode) with WAN.
- You are using fp8_scaled model weights.
- LoRA effects are visibly absent from the output.

If you have already switched to GGUF Q6_K weights, this node is not needed. It can remain in the pipeline with only_if_single_frame=true without affecting multi-frame video generation.

12.3 · Polyhedron Pick Frame

A small companion to Frame Inflate: it picks a **single frame** out of a decoded image batch — typically the batch that Frame Inflate produced for a T2I run.



The Pick Frame node — frame_index -1 selects the middle frame.

Name	Type	Description
images	IMAGE (input)	The decoded image batch (e.g. from VAE Decode).
frame_index	INT (widget)	Which frame to pick (0-based). Default: -1 = middle frame .
image	IMAGE (output)	The selected single frame.

Why the middle frame? In inflated T2I runs the sampler converges most cleanly on the central frame — the first frame can carry anchor artifacts and the last can be slightly blurred by motion continuity. Out-of-range indexes are clamped; an empty batch passes through unchanged with a console warning.

Typical wiring: `Frame Inflate` → `sampler` → `VAE Decode` → `Pick Frame` → `Save Image`

13 · Sigma Nodes

13.1 · What are sigmas? A plain-language explanation

Diffusion models generate images by starting from pure random noise and gradually removing noise step by step until a clean image emerges. **Sigmas** are simply the noise levels at each step — they tell the model how much noise is still in the image at any given point.

Think of it like developing a photograph in a darkroom: you start in complete darkness (high sigma = lots of noise) and slowly bring up the lights (low sigma = little noise) until the image is fully visible. The schedule is the plan for how fast you bring up the lights.

13.2 · The scheduler

The scheduler decides the **shape** of the sigma curve — how quickly the noise level drops across the steps. Different schedulers produce different curves, which leads to different visual results even with the same model and prompt:

karras

The most widely used schedule. Spends more steps in the mid-noise range, which tends to produce well-structured images with good detail. A safe default for most workflows.

exponential

Drops noise faster in the early steps, slower in the late steps. Can produce sharper results but is more sensitive to CFG and step count.

simple / linear

Equal noise reduction per step. Fast and predictable, but often less detailed than karras. Used in some Lightning/distilled models.

bong_tangent / res_multistep and others

Community schedulers with custom curve shapes. `bong_tangent` front-loads the high-noise steps; `res_multistep` uses residual logic for smoother transitions. Results vary by model — worth experimenting with.

13.3 · The sampler

If the scheduler decides what noise levels to use at each step, the **sampler** decides how to move from one noise level to the next. It is the mathematical solver that performs the actual denoising calculation.

A practical analogy: the scheduler is the map (where you go), the sampler is the vehicle (how you get there).

euler

The simplest solver. Fast, consistent, slightly soft at low step counts. Good general-purpose default.

euler_a (ancestral)

Adds a small amount of random noise at each step, producing more varied and sometimes more creative results. Not fully deterministic — the same seed gives slightly different results each run.

dpm++ / dpm++_2m

Higher-order solvers that take larger, smarter steps. Generally produces sharper results than euler at the same step count. dpm++_2m uses two-step momentum — good balance of speed and quality.

uni_pc

Unified Predictor-Corrector. Fast convergence, good for low step counts. The default in WanMoeKSampler workflows.

lcm / ddim

Designed for distilled / Lightning models. lcm is optimised for 4-8 step generation. ddim is deterministic and good for inpainting workflows.

Scheduler + Sampler together

The two settings are independent but interact. A karras schedule with dpm++_2m is a classic high-quality pairing. A simple schedule with lcm is the standard Lightning setup. For WAN 2.2 with WanMoeKSampler the recommended defaults are simple + uni_pc for Lightning LoRAs, or karras + dpm++ for full-step generation.

13.4 · Rho (ρ) — the shape parameter

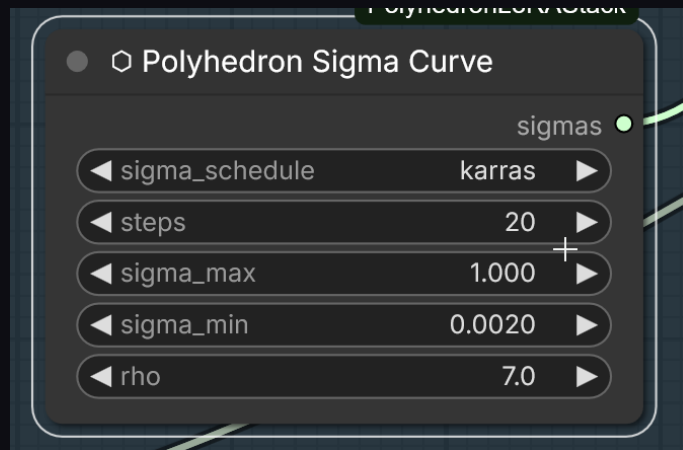
The karras schedule has one additional parameter: **rho**. It controls how curved the sigma schedule is.

- **rho = 1.0** — linear. Equal noise reduction per step.
- **rho = 3.0-5.0** — moderate curve. More steps in the mid-noise range.
- **rho = 7.0** — default. Strong curve, front-loads structure formation.
- **rho > 10.0** — very steep. Most steps are spent at low noise (detail), structure forms quickly.

Higher rho = more detail steps, but less time for large-scale structure. Lower rho = more balanced. The default of 7.0 works well for most cases.

13.5 · Polyhedron Sigma Curve

A standalone sigma schedule generator. It produces a single sigma sequence using a configurable schedule and rho value, replacing the standard BasicScheduler node. Use this in single-model workflows where you want precise control over the schedule shape.



The Polyhedron Sigma Curve node with default values: karras schedule, 20 steps, sigma_max 1.000, sigma_min 0.0020, rho 7.0.

Name	Type	Description
sigma_schedule	COMBO (widget)	Schedule type: karras, exponential, simple, and others.
steps	INT (widget)	Total number of denoising steps. Default: 20.
sigma_max	FLOAT (widget)	Starting noise level. Default: 1.000 (full noise).
sigma_min	FLOAT (widget)	Ending noise level. Default: 0.0020 (near-clean).
rho	FLOAT (widget)	Curve shape. Default: 7.0. See section 13.4.
sigmas	SIGMAS (output)	The computed sigma sequence, ready for the sampler.

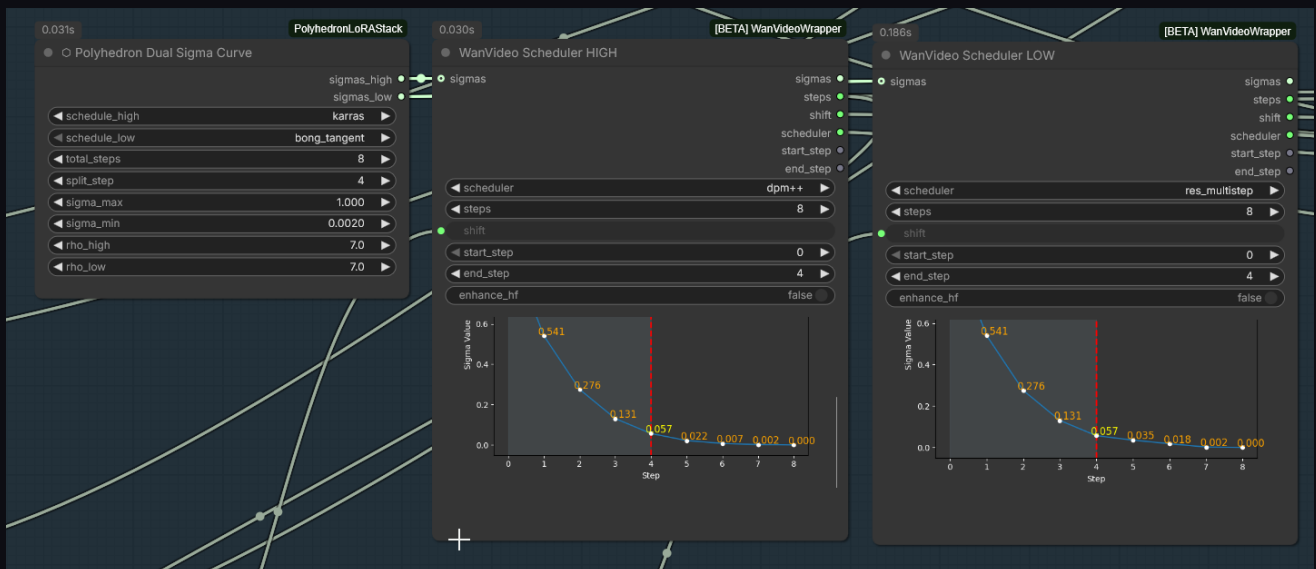
13.6 · Polyhedron Dual Sigma Curve

An extended sigma generator for WAN 2.2 dual-noise workflows. It produces **two separate sigma sequences** — one for the HIGH noise model and one for the LOW noise model — split at a configurable step. Each half uses its own schedule and rho value, giving independent control over the structure-forming phase and the detail-refinement phase.

Name	Type	Description
schedule_high	COMBO (widget)	Schedule for the HIGH noise segment (e.g. karras).
schedule_low	COMBO (widget)	Schedule for the LOW noise segment (e.g. bong_tangent).
total_steps	INT (widget)	Total steps across both segments combined.
split_step	INT (widget)	Step at which HIGH ends and LOW begins. Should match WanMoeKSampler boundary.
sigma_max	FLOAT (widget)	Starting noise level. Default: 1.000.
sigma_min	FLOAT (widget)	Ending noise level. Default: 0.0020.
rho_high	FLOAT (widget)	Curve shape for HIGH segment. Default: 7.0.
rho_low	FLOAT (widget)	Curve shape for LOW segment. Default: 7.0.
sigmas_high	SIGMAS (output)	Sigma sequence for the HIGH noise model.
sigmas_low	SIGMAS (output)	Sigma sequence for the LOW noise model.

13.7 · Connecting to WanVideo Scheduler nodes

In a WAN 2.2 dual-noise pipeline with WanVideoWrapper, the Dual Sigma Curve outputs connect to separate **WanVideo Scheduler HIGH** and **WanVideo Scheduler LOW** nodes. Each Scheduler node displays its sigma curve as a graph with the step values annotated — the red vertical line marks the split point.



Dual Sigma Curve (left) feeding two WanVideo Scheduler nodes — HIGH (centre) and LOW (right). Each Scheduler renders its segment as an annotated sigma graph; the red vertical line marks the split step.

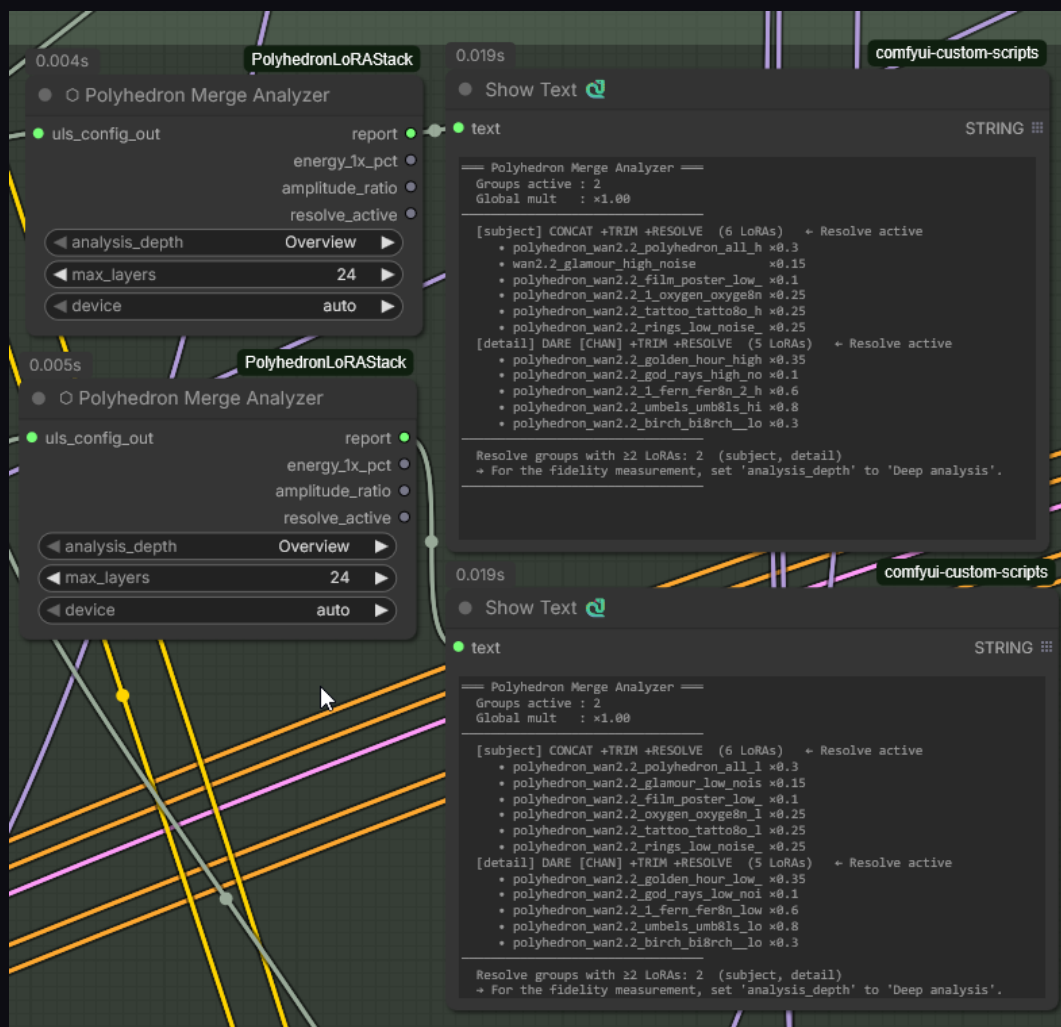
Known issue — LOW sigma chain

In the current release, the LOW sigma segment does not always produce visible refinement under certain scheduler combinations (e.g. `res_multistep` on the LOW path). This is an upstream compatibility issue under investigation. The node is safe to use — it will not corrupt output — but its effect on the LOW path may be neutral until a fix is published.

14 · Polyhedron Merge Analyzer

The Merge Analyzer is a **passive measurement node** for the Stack's Combined / Smooth Mix / Resolve merge path. It reads the Stack's `uls_config_out`, never patches the model, and answers two questions: *what* is the Stack about to merge, and *how faithful* is the Resolve re-pack for the groups that use it.

Use **one Analyzer per Stack** — in a dual-noise workflow that means one for HIGH and one for LOW.



One Merge Analyzer per Stack (HIGH and LOW), each in Overview mode — Stack's `uls_config_out` in, report into a Show Text node.

14.1 · Inputs, outputs and widgets

Name	Type	Description
uls_config_out	STRING (input)	Connect from the Stack's uls_config_out output.
analysis_depth	COMBO (widget)	Overview = instant (selection/modes only). Deep analysis = loads the LoRAs and measures (slower).
max_layers	INT (widget)	Deep analysis: how many of the largest-contribution layers are fully measured. Default: 24.
device	COMBO (widget)	auto = GPU if free (like the real Resolve path), else CPU. cpu forces CPU.
report	STRING (output)	Full formatted report — connect to a Show Text node.
energy_1x_pct	FLOAT (output)	Energy retained at the current rank, aggregated (%).
amplitude_ratio	FLOAT (output)	Amplitude of the re-pack vs. the true sum (1.0 = identical).
resolve_active	BOOLEAN (output)	True when at least one Resolve group with ≥ 2 LoRAs is active.

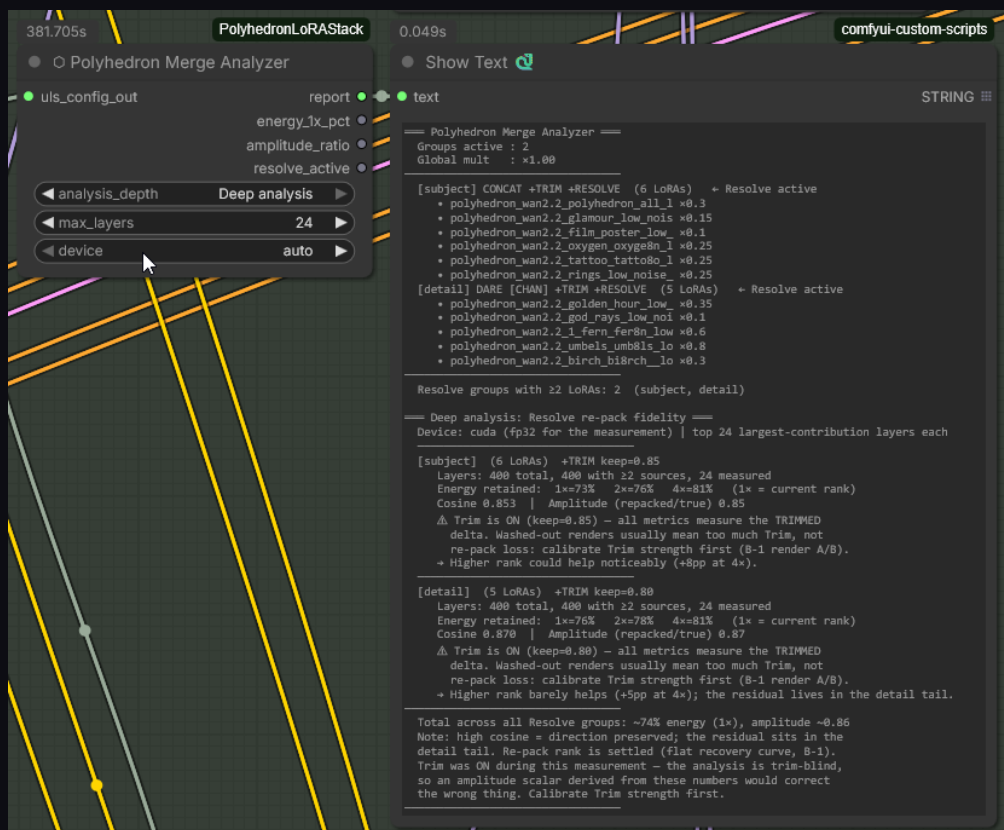
14.2 · Overview mode

Overview is instant. The report lists every active group with its full mode tag (**SEQ** / **CONCAT** / **DARE** [CHAN/ELEM] plus **+TRIM** / **+RESOLVE**), the LoRAs and their weights, and flags groups where Resolve will actually do work ("← Resolve active", i.e. Resolve enabled and ≥ 2 LoRAs). The bottom line tells you which groups can be measured — switch **analysis_depth** to **Deep analysis** for that.

14.3 · Deep analysis

Deep analysis loads the group's LoRAs and measures the Resolve re-pack against the true merged result, per group:

- **Layers** — how many layers exist, how many have ≥ 2 competing sources, how many were measured (capped by **max_layers**, ranked by contribution).
- **Energy retained at 1× / 2× / 4×** — how much of the merged signal survives at the current rank (1×) and what a bigger re-pack would recover (2×/4×).
- **Cosine** — direction match (≈ 0.9 means the merge points the right way).
- **Amplitude (repacked/true)** — overall loudness of the re-pack ($0.90 = \sim 10\%$ quieter).
- A short per-group **verdict** plus a total across all Resolve groups.



Deep analysis report — energy curve, cosine, amplitude, and the Trim warning.

Reading the numbers

A *flat* energy curve (only a few points gained from 1x to 4x) means a higher re-pack rank is not worth it: the residual lives in the fine-detail tail. High cosine = the direction is preserved.

⚠ Trim and the metrics

With Trim ON, all metrics measure the *trimmed* delta (the report warns inline). Washed-out renders are usually **too much Trim**, not re-pack loss — calibrate the Trim strength first (section 3.7).

14.4 · Performance, caching and cancel

- The result is **cached** — the analysis re-runs only when its inputs change, not on every queue.
- Deep analysis prints **live progress** to the console while it works: `[PLS] ANALYZE [group] layer i/N dev layer=...s cum=...s`.
- ComfyUI's red × **cancels** a running deep analysis promptly.
- `device auto` uses the GPU exactly like the real Resolve path; `cpu` forces CPU (slower, but VRAM-free).

```
[PLS] ANALYZE [subject] layer 1/24 cuda layer=8.06s cum=8.1s
[PLS] ANALYZE [subject] layer 5/24 cuda layer=8.13s cum=39.9s
[PLS] ANALYZE [subject] layer 10/24 cuda layer=8.11s cum=80.2s
[PLS] ANALYZE [subject] layer 15/24 cuda layer=7.79s cum=119.3s
[PLS] ANALYZE [subject] layer 20/24 cuda layer=7.77s cum=158.3s
[PLS] ANALYZE [subject] layer 24/24 cuda layer=7.83s cum=189.7s
[PLS] ANALYZE [detail] layer 1/24 cuda layer=7.75s cum=7.7s
[PLS] ANALYZE [detail] layer 5/24 cuda layer=7.93s cum=39.0s
[PLS] ANALYZE [detail] layer 10/24 cuda layer=7.88s cum=78.6s
[PLS] ANALYZE [detail] layer 15/24 cuda layer=7.79s cum=118.2s
[PLS] ANALYZE [detail] layer 20/24 cuda layer=7.98s cum=157.3s
[PLS] ANALYZE [detail] layer 24/24 cuda layer=7.70s cum=188.7s
```

Live [PLS] ANALYZE progress lines in the console during a deep analysis.

© Polyhedron · All rights reserved

Civitai: civitai.com/user/Polyhedron_AI · Patreon: patreon.com/c/polyhedron_ai